

Chapter 8 — Custom Comparator

0. What You Will Learn in This Chapter

Custom Comparator is a sorting sub-pattern, but it deserves its own chapter because many interview problems are not solved by simply sorting ascending or descending. Sometimes the problem asks you to sort by:

- end time
- start time
- height
- frequency
- distance from origin
- lexicographic content
- multiple keys
- special concatenation order
- custom priority rule
- decreasing first key and increasing second key
- greedy-specific ordering

A custom comparator tells C++:

When comparing two elements a and b, should a come before b?

By the end of this chapter, you should understand:

- What a comparator is
- Why custom comparators are needed
- How to write lambdas in C++
- How to sort by one key
- How to sort by multiple keys
- How to sort ascending/descending combinations
- How comparator mistakes break sorting
- What strict weak ordering means
- How to avoid overflow in comparator
- How to sort vectors, pairs, strings, intervals, and custom objects
- How custom comparators enable greedy solutions
- How custom comparators are used in priority_queue
- How to recognize custom comparator problems

Core LeetCode problems covered in this chapter:

- 1.1. K Closest Points to Origin
- 2.2. Largest Number
- 3.3. Reorder Data in Log Files
- 4.4. Merge Intervals, comparator version
- 5.5. Non-overlapping Intervals
- 6.6. Minimum Number of Arrows to Burst Balloons
- 7.7. Queue Reconstruction by Height
- 8.8. Sort Characters By Frequency
- 9.9. Top K Frequent Words
- 10.10. Top K Frequent Elements
- 11.11. Meeting Rooms II, heap comparator preview
- 12.12. Car Pooling / event sorting preview
- 13.13. Remove Covered Intervals
- 14.14. Russian Doll Envelopes
- 15.15. Maximum Units on a Truck

1. What Is a Custom Comparator?

A custom comparator is a function that defines how two elements should be ordered.
Default sorting:

```
sort(nums.begin(), nums.end());
```

sorts numbers ascending.

Example:

```
vector<int> nums = {5, 1, 3};  
sort(nums.begin(), nums.end());
```

Result:

```
{1, 3, 5}
```

But what if you want to sort intervals by end time?

```
intervals = {{1,3}, {2,2}, {4,5}}
```

Default sort sorts by first value, then second value.

But to sort by end time, you need:

```
sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>& b) {  
    return a[1] < b[1];  
});
```

This lambda says:

Put interval a before interval b if a ends earlier.

2. Why Custom Comparators Exist

Many problems require an order that is not the default order.

Default C++ sorting gives:

- numbers ascending
- strings lexicographically ascending
- pairs sorted by first, then second
- vectors sorted lexicographically

But interview problems often need something else.

Examples:

K Closest Points	distance ascending
Largest Number	$a + b > b + a$
Non-overlapping Intervals	end ascending
Queue Reconstruction	height descending, k ascending

Problem	Needed Order
Top K Frequent Words	frequency descending, word ascending
Reorder Logs	content ascending, identifier tie-break
Remove Covered Intervals	start ascending, end descending

Custom comparator exists because the sorting order must match the problem's logic.

3. Core Need / Why Use a Custom Comparator?

Use a custom comparator when natural order is not enough.

3.1 Why Not Default Sort?

Default sort may sort by the wrong key.

Example:

```
intervals = {{1,10}, {2,3}, {4,5}}
```

Default sort by start:

```
{{1,10}, {2,3}, {4,5}}
```

But for Non-overlapping Intervals, you need earliest end:

```
{{2,3}, {4,5}, {1,10}}
```

So default sort is wrong.

3.2 Why Not Sort Then Manually Rearrange?

That is usually more complicated and slower.

It is better to encode the correct order directly into comparator.

3.3 Why Not Use Multiple Sorts?

Sometimes people sort once by one key, then again by another key.

This can be wrong unless using `stable_sort` carefully.

Better:

```
sort(arr.begin(), arr.end(), [](auto& a, auto& b) {
    if (a.key1 != b.key1) {
        return a.key1 < b.key1;
    }

    return a.key2 > b.key2;
});
```

One comparator with clear tie-breakers is usually best.

4. When to Use Custom Comparator

Use custom comparator when problem says or implies:

4.1 Sort by a Specific Field

Examples:

- sort intervals by end
- sort people by height
- sort points by distance
- sort boxes by units per box

Problems:

- Non-overlapping Intervals
 - Queue Reconstruction
 - K Closest Points
 - Maximum Units on a Truck
-

4.2 Sort by Multiple Keys

Example:

Queue Reconstruction:

- height descending
- if same height, k ascending

```
if (a[0] != b[0]) {  
    return a[0] > b[0];  
}  
return a[1] < b[1];
```

4.3 Sort by Frequency

Examples:

- Sort Characters By Frequency
 - Top K Frequent Elements
 - Top K Frequent Words
-

4.4 Sort by Custom String Logic

Examples:

- Largest Number
 - Reorder Data in Log Files
-

4.5 Sort Events

Sweep line problems often require event ordering.

Example:

At the same time, should end event come before start event?

Meeting Rooms style:

```
{time, delta}
```

End event -1 should happen before start event +1 at same time.
Default pair sorting handles this because $-1 < +1$.

4.6 Sort for Greedy

Many greedy algorithms depend on the correct sorted order.

Examples:

- earliest ending interval
 - most units per box
 - tallest person first
 - smallest greed first
 - smallest ending arrow point first
-

5. When NOT to Use a Custom Comparator

5.1 Default Sort Already Works

If sorting numbers ascending is enough:

```
sort(nums.begin(), nums.end());
```

No comparator needed.

5.2 Original Order Must Be Preserved

If the original order matters, sorting may be wrong.

Example:

- Stock prices
 - Maximum Subarray
 - subsequence problems
-

5.3 Need Stable Ordering of Equal Elements

If equal elements must keep original order, use:

```
stable_sort(...)
```

Example:

- Reorder Data in Log Files for digit logs
-

5.4 Need Dynamic Ordering

If elements arrive dynamically and you repeatedly need min/max/top, use:

- heap
- ordered set
- multiset

rather than sorting repeatedly.

6. C++ Comparator Basics

A comparator returns true if a should come before b.

```
[](const auto& a, const auto& b) {  
    return a < b;  
}
```

This sorts ascending.

Descending:

```
[](const auto& a, const auto& b) {  
    return a > b;  
}
```

6.1 Lambda Syntax

```
sort(arr.begin(), arr.end(), [](const int& a, const int& b) {  
    return a < b;  
});
```

Breakdown:

[]

Capture list.

(const int& a, const int& b)

Parameters.

{return a < b;}

Comparison logic.

6.2 Use References

Prefer:

const vector<int>& a

instead of:

vector<int> a

because passing by value copies the vector.

Good:

```
sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>&  
b) {  
    return a[1] < b[1];  
});
```

7. Strict Weak Ordering

This is very important.

A comparator must define a valid ordering.

For equal values, comparator should return false.

Wrong:

```
return a[0] <= b[0];
```

Why wrong?

If $a[0] == b[0]$, then:

```
comp(a, b) == true
```

```
comp(b, a) == true
```

This breaks sorting rules.

Correct:

```
return a[0] < b[0];
```

7.1 Bad Comparator Example

Wrong:

```
sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>& b) {  
    return a[1] <= b[1];  
});
```

Correct:

```
sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>& b) {  
    return a[1] < b[1];  
});
```

7.2 Comparator Should Be Consistent

If a should come before b, and b should come before c, then a should come before c.

Do not write comparators with random or changing behavior.

Bad:

```
return rand() % 2;
```

Never do this.

8. Avoid Overflow in Comparator

Wrong:

```
return a[1] - b[1] < 0;
```

If values are large, subtraction can overflow.

Correct:

```
return a[1] < b[1];
```

For distance:

Potential overflow:

```
int dist = x * x + y * y;
```

If x and y can be large, use long long:

```
long long distA = 1LL * a[0] * a[0] + 1LL * a[1] * a[1];
```

9. Core Comparator Templates

Template 1 — Sort Ascending by One Key

```
sort(arr.begin(), arr.end(), [](const auto& a, const auto& b) {  
    return a.key < b.key;  
});
```

For vector:

```
sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>&  
b) {  
    return a[0] < b[0];  
});
```

Template 2 — Sort Descending by One Key

```
sort(arr.begin(), arr.end(), [](const auto& a, const auto& b) {  
    return a.key > b.key;  
});
```

Example:

```
sort(boxTypes.begin(), boxTypes.end(), [](const vector<int>& a, const  
vector<int>& b) {  
    return a[1] > b[1];  
});
```

Template 3 — Sort by Two Keys

Ascending first key, ascending second key:

```
sort(arr.begin(), arr.end(), [](const vector<int>& a, const vector<int>& b) {  
    if (a[0] != b[0]) {  
        return a[0] < b[0];  
    }  
  
    return a[1] < b[1];  
});
```

Template 4 — Ascending First Key, Descending Second Key

```
sort(arr.begin(), arr.end(), [](const vector<int>& a, const vector<int>& b) {  
    if (a[0] != b[0]) {  
        return a[0] < b[0];  
    }  
  
    return a[1] > b[1];  
});
```

Used for:

- Remove Covered Intervals
- Russian Doll Envelopes, with width asc, height desc

Template 5 — Descending First Key, Ascending Second Key

```
sort(arr.begin(), arr.end(), [](const vector<int>& a, const vector<int>& b) {  
    if (a[0] != b[0]) {  
        return a[0] > b[0];  
    }  
  
    return a[1] < b[1];  
});
```

Used for:

- Queue Reconstruction by Height

Template 6 — Sort by Distance

```
sort(points.begin(), points.end(), [](const vector<int>& a, const vector<int>& b) {  
    long long distA = 1LL * a[0] * a[0] + 1LL * a[1] * a[1];  
    long long distB = 1LL * b[0] * b[0] + 1LL * b[1] * b[1];  
  
    return distA < distB;  
});
```

Template 7 — Sort Strings by Custom Concatenation

```
sort(words.begin(), words.end(), [](const string& a, const string& b) {  
    return a + b > b + a;  
});
```

Used for:

- Largest Number

Template 8 — Sort by Frequency

```
sort(arr.begin(), arr.end(), [](const pair<char, int>& a, const pair<char, int>& b) {  
    return a.second > b.second;  
});
```

Template 9 — Frequency Descending, Word Ascending

```
sort(words.begin(), words.end(), [](const pair<string, int>& a, const pair<string,  
int>& b) {  
    if (a.second != b.second) {  
        return a.second > b.second;  
    }  
  
    return a.first < b.first;  
});
```

Used for:

- Top K Frequent Words

Template 10 — Event Sorting

```
vector<pair<int, int>> events;  
  
events.push_back({start, 1});  
events.push_back({end, -1});  
  
sort(events.begin(), events.end());
```

At same time, -1 comes before +1.

Useful for meeting-room logic where end should be processed before start.

10. Comparator for priority_queue

Sorting comparator and heap comparator feel opposite at first.

Default:

```
priority_queue<int> pq;
```

is max heap.

Min heap:

```
priority_queue<int, vector<int>, greater<int>> pq;
```

10.1 Custom Comparator Struct

```
struct Compare {  
    bool operator()(const vector<int>& a, const vector<int>& b) {  
        return a[1] > b[1];  
    }
```

```
}  
};
```

Used like:

```
priority_queue<vector<int>, vector<vector<int>>, Compare> pq;
```

Important:

For priority_queue, comparator returns true if a has lower priority than b.

This feels reversed compared to sort.

10.2 Min Heap by First Value

```
priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;
```

Pairs are compared lexicographically.

11. Major Custom Comparator Patterns

Sort by end	$a[1] < b[1]$	Non-overlapping Intervals
Sort by start	$a[0] < b[0]$	Merge Intervals
Sort by distance	$\text{distA} < \text{distB}$	K Closest Points
Sort by frequency	freq desc	Sort Characters By Frequency
Sort by frequency and lexicographic	freq desc, word asc	Top K Frequent Words
Sort by concatenation	$a+b > b+a$	Largest Number
Height desc, k asc	h desc, k asc	Queue Reconstruction
Width asc, height desc	w asc, h desc	Russian Doll Envelopes
Start asc, end desc	start asc, end desc	Remove Covered Intervals
Units desc	unitsPerBox desc	Maximum Units on a Truck

12. Full Problem Walkthrough 1 — LeetCode 973: K Closest Points to Origin

12.1 Problem Summary

Given points on a 2D plane, return the k closest points to origin (0,0).

Example:

```
points = {{1,3}, {-2,2}}
k = 1
```

Output:

```
{{-2,2}}
```

Because distance squared:

```
{1,3}: 1*1 + 3*3 = 10
{-2,2}: 4 + 4 = 8
```

12.2 How to Identify Custom Comparator

Problem asks:

- sort/find by distance
- distance is not default ordering

Use custom comparator based on squared distance.

12.3 Brute Force Idea

Compute distance for every point and repeatedly find smallest.

Could be $O(k * n)$.

12.4 Sorting Intuition

Sort all points by distance ascending.

Return first k.

12.5 C++ Code

```
class Solution {
public:
    vector<vector<int>> kClosest(vector<vector<int>>& points, int k) {
        sort(points.begin(), points.end(), [](const vector<int>& a, const vector<int>&
b) {
            long long distA = 1LL * a[0] * a[0] + 1LL * a[1] * a[1];
            long long distB = 1LL * b[0] * b[0] + 1LL * b[1] * b[1];

            return distA < distB;
        });
    }
};
```

```

    });

    return vector<vector<int>>(points.begin(), points.begin() + k);
}
};

```

12.6 Code Explanation

```
long long distA = 1LL * a[0] * a[0] + 1LL * a[1] * a[1];
```

Uses squared distance.

No need for:

```
sqrt(...)
```

because square root preserves order.

12.7 Dry Run

```
points = {{1,3}, {-2,2}, {2,-2}}
```

```
k = 2
```

Distances:

{1,3}	10
{-2,2}	8
{2,-2}	8

Sorted by distance:

```
{{-2,2}, {2,-2}, {1,3}}
```

Return first two.

12.8 Complexity

Time: $O(n \log n)$

Space: $O(k)$ output.

12.9 Alternatives

- Max heap of size k : $O(n \log k)$
- Quickselect: average $O(n)$

Sorting is simplest.

12.10 Edge Cases

- k equals number of points
- negative coordinates
- same distance
- origin point $\{0,0\}$

12.11 Common Mistakes

Mistake 1: Using actual distance with sqrt

Unnecessary.

Mistake 2: Overflow in distance

Use long long if coordinates are large.

12.12 Similar Problems

- Kth Largest Element
 - Top K Frequent Elements
 - Find K Closest Elements
-

13. Full Problem Walkthrough 2 — LeetCode 179: Largest Number

13.1 Problem Summary

Given a list of non-negative integers, arrange them to form the largest possible number.

Example:

```
nums = {3, 30, 34, 5, 9}
```

Output:

```
"9534330"
```

13.2 How to Identify Custom Comparator

Problem asks for special ordering of numbers as strings.

Normal numeric sort fails.

Example:

```
3 and 30
```

Compare:

```
"3" + "30" = "330"
```

```
"30" + "3" = "303"
```

Since "330" is larger, 3 should come before 30.

13.3 Brute Force Idea

Try all permutations and choose largest.

Time: $O(n!)$, impossible.

13.4 Comparator Intuition

For two numbers represented as strings a and b:

Put a before b if:

```
a + b > b + a
```

13.5 C++ Code

```
class Solution {
public:
    string largestNumber(vector<int>& nums) {
        vector<string> words;

        for (int x : nums) {
            words.push_back(to_string(x));
        }

        sort(words.begin(), words.end(), [](const string& a, const string& b) {
            return a + b > b + a;
        });

        if (words[0] == "0") {
            return "0";
        }

        string result;

        for (string& word : words) {
            result += word;
        }

        return result;
    }
};
```

13.6 Code Explanation

```
to_string(x)
```

Converts integer to string.

```
return a + b > b + a;
```

Determines which order creates larger combined number.

```
if (words[0] == "0")
```

If largest string is "0", all are zero.

Return "0" instead of "000".

13.7 Dry Run

nums = {3, 30, 34, 5, 9}

Key comparisons:

9 before 5
5 before 34
34 before 3
3 before 30

Sorted:

{"9", "5", "34", "3", "30"}

Result:

"9534330"

13.8 Complexity

Let:

- n = number of integers
- m = average digit length

Comparator costs $O(m)$.

Sorting:

$O(n \log n * m)$

Space:

$O(n * m)$

13.9 Edge Cases

- all zeroes
- single number
- different digit lengths
- repeated numbers

13.10 Common Mistakes

Mistake 1: Numeric descending sort

Fails for:

{3, 30}

Mistake 2: Forgetting all-zero case

Wrong output:

"00"

Correct:

"0"

Mistake 3: Comparator using $\geq$

Wrong:

```
return a + b >= b + a;
```

Correct:

```
return a + b > b + a;
```

13.11 Similar Problems

- Reorder Data in Log Files
- Custom string sorting
- Smallest Number variants

14. Full Problem Walkthrough 3 — LeetCode 937: Reorder Data in Log Files

14.1 Problem Summary

You are given logs.

Each log has:

```
identifier content
```

There are two types:

- letter-logs: content starts with a letter
- digit-logs: content starts with a digit

Rules:

2. Letter-logs come before digit-logs.
3. Letter-logs sorted lexicographically by content.
4. If content ties, sort by identifier.
5. Digit-logs remain in original order.

14.2 How to Identify Custom Comparator

Problem gives very specific ordering rules.

Need custom comparator.

Need `stable_sort` to preserve digit-log relative order.

14.3 C++ Code

```
class Solution {
public:
    vector<string> reorderLogFiles(vector<string>& logs) {
        stable_sort(logs.begin(), logs.end(), [](const string& a, const string& b) {
            int posA = a.find(' ');
            int posB = b.find(' ');

            string idA = a.substr(0, posA);
```

```

    string idB = b.substr(0, posB);

    string contentA = a.substr(posA + 1);
    string contentB = b.substr(posB + 1);

    bool digitA = isdigit(contentA[0]);
    bool digitB = isdigit(contentB[0]);

    if (!digitA && !digitB) {
        if (contentA != contentB) {
            return contentA < contentB;
        }

        return idA < idB;
    }

    if (!digitA && digitB) {
        return true;
    }

    if (digitA && !digitB) {
        return false;
    }

    return false;
});

return logs;
}
};

```

14.4 Code Explanation

stable_sort

Keeps original order for equivalent elements.
This matters for digit logs.

a.find(' ')

Finds separator between identifier and content.

isdigit(contentA[0])

Determines log type.

14.5 Comparator Cases

letter	letter	compare content, then id
letter	digit	true
digit	letter	false
digit	digit	false, stable_sort keeps order

14.6 Complexity

Let:

- n = number of logs
- m = average log length

Time:

$O(n \log n * m)$

Space depends on string copies and sorting.

14.7 Edge Cases

- all digit logs
 - all letter logs
 - same content different identifiers
 - mixed logs
 - multiple spaces not usually issue under constraints
-

14.8 Common Mistakes

- sorting digit logs
 - forgetting identifier tie-breaker
 - using normal sort and expecting digit logs to preserve order
 - comparing entire log instead of content first
-

14.9 Similar Problems

- Largest Number
 - Sort by custom string keys
 - Log sorting problems
-

15. Full Problem Walkthrough 4 — LeetCode 435: Non-overlapping Intervals

15.1 Problem Summary

Given intervals, return minimum number to remove so remaining intervals do not overlap.

Example:

```
intervals = {{1,2}, {2,3}, {3,4}, {1,3}}
```

Output:

```
1
```

15.2 How to Identify Custom Comparator

This is greedy interval scheduling.

Need sort by end time ascending.

Default sort by start is not ideal.

15.3 C++ Code

```
class Solution {
public:
    int eraseOverlapIntervals(vector<vector<int>>& intervals) {
        if (intervals.empty()) {
            return 0;
        }

        sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const
vector<int>& b) {
            return a[1] < b[1];
        });

        int removals = 0;
        int lastEnd = intervals[0][1];

        for (int i = 1; i < intervals.size(); i++) {
            if (intervals[i][0] < lastEnd) {
                removals++;
            } else {
                lastEnd = intervals[i][1];
            }
        }

        return removals;
    }
};
```

15.4 Why Sort by End?

To keep maximum number of intervals, always choose the interval that ends earliest. This leaves maximum room for future intervals.

15.5 Complexity

Time: $O(n \log n)$

Space: $O(1)$ extra.

15.6 Common Mistakes

- sorting by start without careful logic
 - treating touching intervals as overlapping
 - comparator using $\leq$
-

15.7 Similar Problems

- Minimum Number of Arrows
 - Maximum Length of Pair Chain
 - Activity Selection
-

16. Full Problem Walkthrough 5 — LeetCode 452: Minimum Number of Arrows to Burst Balloons

16.1 Problem Summary

Each balloon is interval $[start, end]$.

One arrow at position x bursts all balloons where:

$start \leq x \leq end$

Return minimum arrows.

16.2 How to Identify Custom Comparator

This is interval coverage.

Sort by end time ascending.

Shoot arrow at earliest ending balloon's end.

16.3 C++ Code

```
class Solution {
public:
    int findMinArrowShots(vector<vector<int>>& points) {
        if (points.empty()) {
            return 0;
        }

        sort(points.begin(), points.end(), [](const vector<int>& a, const vector<int>&
```

```
b) {  
    return a[1] < b[1];  
};  
  
int arrows = 1;  
long long arrowPos = points[0][1];  
  
for (int i = 1; i < points.size(); i++) {  
    if (points[i][0] > arrowPos) {  
        arrows++;  
        arrowPos = points[i][1];  
    }  
}  
  
return arrows;  
}
```

16.4 Important Comparator Note

Do not write:

```
return a[1] - b[1] < 0;
```

Balloon coordinates can be large.

Use:

```
return a[1] < b[1];
```

16.5 Complexity

Time: $O(n \log n)$

Space: $O(1)$ extra.

16.6 Similar Problems

- Non-overlapping Intervals
- Meeting Rooms
- Merge Intervals

17. Full Problem Walkthrough 6 — LeetCode 406: Queue Reconstruction by Height

17.1 Problem Summary

Each person is:

```
{height, k}
```

where k is number of people in front with height greater than or equal to them.
Reconstruct queue.

17.2 How to Identify Custom Comparator

Need special ordering.

Taller people affect shorter people's k.

Shorter people do not affect taller people's k.

So sort:

6. height descending
 7. k ascending
-

17.3 C++ Code

```
class Solution {
public:
    vector<vector<int>> reconstructQueue(vector<vector<int>>& people) {
        sort(people.begin(), people.end(), [](const vector<int>& a, const vector<int>&
b) {
            if (a[0] != b[0]) {
                return a[0] > b[0];
            }

            return a[1] < b[1];
        });

        vector<vector<int>> result;

        for (auto& person : people) {
            result.insert(result.begin() + person[1], person);
        }

        return result;
    }
};
```

17.4 Why This Comparator?

For taller people, their k count depends only on people at least as tall.

So place taller people first.

For same height, smaller k should come earlier.

17.5 Dry Run

Input:

{{7,0}, {4,4}, {7,1}, {5,0}, {6,1}, {5,2}}

Sorted:

{{7,0}, {7,1}, {6,1}, {5,0}, {5,2}, {4,4}}

Insert by k:

{7,0}	{{7,0}}
{7,1}	{{7,0},{7,1}}
{6,1}	{{7,0},{6,1},{7,1}}
{5,0}	{{5,0},{7,0},{6,1},{7,1}}
{5,2}	{{5,0},{7,0},{5,2},{6,1},{7,1}}
{4,4}	{{5,0},{7,0},{5,2},{6,1},{4,4},{7,1}}

17.6 Complexity

Sorting:

$O(n \log n)$

Vector insertions:

$O(n^2)$

Total:

$O(n^2)$

Space:

$O(n)$

17.7 Common Mistakes

- sorting height ascending
- sorting same height by k descending
- using push_back
- not understanding why shorter people do not affect taller people

17.8 Similar Problems

- Russian Doll Envelopes
- Greedy ordering problems
- Custom sort + insert problems

18. Full Problem Walkthrough 7 — LeetCode 451: Sort Characters By Frequency

18.1 Problem Summary

Given string *s*, sort characters by decreasing frequency.

Example:

```
s = "tree"
```

Output could be:

```
"eert"
```

or:

```
"eetr"
```

18.2 How to Identify Custom Comparator

Need sorting by frequency, not alphabetically.

Use:

8. frequency map
9. vector of pairs
10. sort by count descending

18.3 C++ Code

```
class Solution {
public:
    string frequencySort(string s) {
        unordered_map<char, int> freq;

        for (char c : s) {
            freq[c]++;
        }

        vector<pair<char, int>> arr;

        for (auto& p : freq) {
            arr.push_back({p.first, p.second});
        }

        sort(arr.begin(), arr.end(), [](const pair<char, int>& a, const pair<char, int>& b) {
            return a.second > b.second;
        });
    }
};
```

```
});

string result;

for (auto& p : arr) {
    char c = p.first;
    int count = p.second;

    result += string(count, c);
}

return result;
};
```

18.4 Code Explanation

```
string(count, c)
```

Creates a string with count copies of character c.

Example:

```
string(3, 'a') = "aaa"
```

18.5 Complexity

Let n be string length and k be unique characters.

Frequency count: $O(n)$

Sort unique chars: $O(k \log k)$

Build result: $O(n)$

Space: $O(k)$

18.6 Edge Cases

- all same characters
- all unique characters
- uppercase and lowercase
- digits/symbols
- ties in frequency, any order usually accepted

18.7 Similar Problems

- Top K Frequent Elements
- Top K Frequent Words
- Reorganize String

19. Full Problem Walkthrough 8 — LeetCode 692: Top K Frequent Words

19.1 Problem Summary

Given list of words and integer k, return k most frequent words.

Ordering:

11. higher frequency first
12. if frequency ties, lexicographically smaller word first

Example:

```
words = {"i","love","leetcode","i","love","coding"}  
k = 2
```

Output:

```
{"i", "love"}
```

19.2 How to Identify Custom Comparator

Need sorting by two keys:

13. frequency descending
14. word ascending

19.3 C++ Code — Sorting Version

```
class Solution {  
public:  
    vector<string> topKFrequent(vector<string>& words, int k) {  
        unordered_map<string, int> freq;  
  
        for (string& word : words) {  
            freq[word]++;  
        }  
  
        vector<pair<string, int>> arr;  
  
        for (auto& p : freq) {  
            arr.push_back({p.first, p.second});  
        }  
  
        sort(arr.begin(), arr.end(), [](const pair<string, int>& a, const pair<string,  
int>& b) {  
            if (a.second != b.second) {  
                return a.second > b.second;  
            }  
        });  
    }  
};
```

```

    }

    return a.first < b.first;
});

vector<string> result;

for (int i = 0; i < k; i++) {
    result.push_back(arr[i].first);
}

return result;
}
};

```

19.4 Comparator Explanation

```

if (a.second != b.second) {
    return a.second > b.second;
}

```

Higher frequency first.

```
return a.first < b.first;
```

Lexicographically smaller word first.

19.5 Complexity

Let:

- n = total words
- u = unique words

Time:

$O(n + u \log u)$

Space:

$O(u)$

19.6 Heap Alternative Preview

Heap can solve in:

$O(n + u \log k)$

But comparator becomes trickier.

We will cover that in Heap chapter.

19.7 Common Mistakes

- tie-breaking in wrong direction

- sorting words alphabetically first
 - forgetting unique words vs total words
 - using `unordered_map` iteration order
-

19.8 Similar Problems

- Top K Frequent Elements
 - Sort Characters By Frequency
 - Reorganize String
-

20. Full Problem Walkthrough 9 — LeetCode 347: Top K Frequent Elements

20.1 Problem Summary

Given integer array `nums`, return `k` most frequent elements.

Example:

```
nums = {1,1,1,2,2,3}
```

```
k = 2
```

Output:

```
{1,2}
```

20.2 How to Identify Custom Comparator

Need order by frequency, not value.

Use frequency map and sort by count descending.

20.3 C++ Code — Sorting Version

```
class Solution {
public:
    vector<int> topKFrequent(vector<int>& nums, int k) {
        unordered_map<int, int> freq;

        for (int x : nums) {
            freq[x]++;
        }

        vector<pair<int, int>> arr;

        for (auto& p : freq) {
            arr.push_back({p.first, p.second});
        }
    }
};
```

```

    sort(arr.begin(), arr.end(), [](const pair<int, int>& a, const pair<int, int>& b) {
        return a.second > b.second;
    });

    vector<int> result;

    for (int i = 0; i < k; i++) {
        result.push_back(arr[i].first);
    }

    return result;
}
};

```

20.4 Complexity

Let u be number of unique values.

Time:

$O(n + u \log u)$

Space:

$O(u)$

20.5 Alternatives

- heap: $O(n + u \log k)$
- bucket sort: $O(n)$

Sorting is simplest.

20.6 Similar Problems

- Top K Frequent Words
- Sort Characters By Frequency
- K Closest Points

21. Full Problem Walkthrough 10 — LeetCode 1710: Maximum Units on a Truck

21.1 Problem Summary

You are given `boxTypes`, where:

`boxTypes[i] = {numberOfBoxes, unitsPerBox}`

Truck can carry at most truckSize boxes.

Return maximum total units.

Example:

```
boxTypes = {{1,3}, {2,2}, {3,1}}
truckSize = 4
```

Output:

8

Take:

- 1 box of 3 units
- 2 boxes of 2 units
- 1 box of 1 unit

Total = 8.

21.2 How to Identify Custom Comparator

Need choose boxes with highest units per box first.

Sort by unitsPerBox descending.

21.3 C++ Code

```
class Solution {
public:
    int maximumUnits(vector<vector<int>>& boxTypes, int truckSize) {
        sort(boxTypes.begin(), boxTypes.end(), [](const vector<int>& a, const
vector<int>& b) {
            return a[1] > b[1];
        });

        int totalUnits = 0;

        for (auto& box : boxTypes) {
            int count = box[0];
            int units = box[1];

            int take = min(count, truckSize);

            totalUnits += take * units;
            truckSize -= take;

            if (truckSize == 0) {
                break;
            }
        }
    }
}
```

```
        return totalUnits;
    }
};
```

21.4 Greedy Explanation

Since every box consumes one truck slot, boxes with more units per box are always better.

So sort descending by units.

21.5 Complexity

Time: $O(n \log n)$

Space: $O(1)$ extra.

21.6 Similar Problems

- Assign Cookies
- Fractional Knapsack style greedy
- Boats to Save People

22. Full Problem Walkthrough 11 — LeetCode 1288: Remove Covered Intervals

22.1 Problem Summary

Given intervals, remove intervals that are covered by another interval.

Interval $[a,b]$ is covered by $[c,d]$ if:

$c \leq a$ and $b \leq d$

Return number of remaining intervals.

Example:

intervals = $\{\{1,4\}, \{3,6\}, \{2,8\}\}$

Output:

2

$\{3,6\}$ is covered by $\{2,8\}$.

22.2 How to Identify Custom Comparator

Need detect covered intervals.

Sort by:

15. start ascending
16. end descending when start ties

Why?

If two intervals have same start, longer one should come first.

22.3 C++ Code

```
class Solution {
public:
    int removeCoveredIntervals(vector<vector<int>>& intervals) {
        sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const
vector<int>& b) {
            if (a[0] != b[0]) {
                return a[0] < b[0];
            }

            return a[1] > b[1];
        });

        int remaining = 0;
        int maxEnd = 0;

        for (auto& interval : intervals) {
            if (interval[1] > maxEnd) {
                remaining++;
                maxEnd = interval[1];
            }
        }

        return remaining;
    }
};
```

22.4 Why End Descending on Tie?

Example:

{1,4}, {1,3}

If sorted end ascending:

{1,3}, {1,4}

You might count {1,3} before seeing it is covered.

End descending:

{1,4}, {1,3}

Then {1,3} is clearly covered.

22.5 Complexity

Time: $O(n \log n)$

Space: $O(1)$ extra.

22.6 Edge Cases

- same start
 - same end
 - all covered
 - none covered
 - nested intervals
-

22.7 Similar Problems

- Merge Intervals
 - Non-overlapping Intervals
 - Minimum Arrows
 - Russian Doll Envelopes
-

23. Full Problem Walkthrough 12 — LeetCode 354: Russian Doll Envelopes

This is harder and combines custom sorting with LIS.

23.1 Problem Summary

You are given envelopes:

`{width, height}`

One envelope can fit into another if both width and height are strictly greater.
Return maximum number of envelopes you can Russian-doll.

Example:

`envelopes = {{5,4}, {6,4}, {6,7}, {2,3}}`

Output:

`3`

Sequence:

`{2,3} -> {5,4} -> {6,7}`

23.2 How to Identify Custom Comparator

Need sort by width ascending.

But if widths are equal, heights should be sorted descending.

Why?

Because equal widths cannot fit into each other.

If heights were ascending for equal widths, LIS on height could incorrectly choose both.

23.3 Sorting Rule

width ascending
height descending if same width

Then run LIS on heights.

23.4 C++ Code

```
class Solution {
public:
    int maxEnvelopes(vector<vector<int>>& envelopes) {
        sort(envelopes.begin(), envelopes.end(), [](const vector<int>& a, const
vector<int>& b) {
            if (a[0] != b[0]) {
                return a[0] < b[0];
            }

            return a[1] > b[1];
        });

        vector<int> tails;

        for (auto& env : envelopes) {
            int h = env[1];

            auto it = lower_bound(tails.begin(), tails.end(), h);

            if (it == tails.end()) {
                tails.push_back(h);
            } else {
                *it = h;
            }
        }

        return tails.size();
    }
};
```

23.5 Why Height Descending for Equal Width?

Example:

{5,4}, {5,5}

These cannot nest because widths are equal.
If sorted by height ascending, LIS might pick 4,5.
Sorting height descending gives:

{5,5}, {5,4}

LIS will not pick both in increasing height order.

23.6 Complexity

Sorting:

$O(n \log n)$

LIS:

$O(n \log n)$

Total:

$O(n \log n)$

Space:

$O(n)$

23.7 Edge Cases

- same width
- same height
- all equal
- already chainable
- no nesting possible

23.8 Similar Problems

- Longest Increasing Subsequence
- Queue Reconstruction by Height
- Maximum Length of Pair Chain

24. Mini Problem 1 — Sort Intervals by Start

```
sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>& b) {  
    return a[0] < b[0];  
});
```

Use for:

- Merge Intervals
- Meeting Rooms
- Insert Interval if unsorted

25. Mini Problem 2 — Sort Intervals by End

```
sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>& b) {  
    return a[1] < b[1];  
});
```

Use for:

- Non-overlapping Intervals
 - Minimum Number of Arrows
 - Activity selection
-

26. Mini Problem 3 — Sort Pair Vector by Second Value

```
vector<pair<int, int>> arr;  
  
sort(arr.begin(), arr.end(), [](const pair<int, int>& a, const pair<int, int>& b) {  
    return a.second < b.second;  
});
```

Descending:

```
sort(arr.begin(), arr.end(), [](const pair<int, int>& a, const pair<int, int>& b) {  
    return a.second > b.second;  
});
```

27. Mini Problem 4 — Sort by Absolute Value

```
sort(nums.begin(), nums.end(), [](int a, int b) {  
    return abs(a) < abs(b);  
});
```

If tie needs natural order:

```
sort(nums.begin(), nums.end(), [](int a, int b) {  
    if (abs(a) != abs(b)) {  
        return abs(a) < abs(b);  
    }  
});
```

```
    return a < b;
};
```

28. Mini Problem 5 — Sort Events Carefully

For sweep line events:

```
vector<pair<int, int>> events;
```

If delta = -1 for end and +1 for start:

```
sort(events.begin(), events.end());
```

At same time, end happens before start because:

```
-1 < +1
```

But if you encode start as 0 and end as 1, tie behavior changes.
Always design event encoding intentionally.

29. Common Edge Cases for Custom Comparator

29.1 Equal Keys

Always decide tie behavior.

Example:

```
if (a[0] != b[0]) {
    return a[0] < b[0];
}
return a[1] > b[1];
```

29.2 Same Start Intervals

For covered intervals, sort end descending.

For merge intervals, default end ascending is fine.

29.3 Same Frequency

For Top K Frequent Words, tie is lexicographic ascending.

For Sort Characters By Frequency, tie usually does not matter.

29.4 All Zeroes

Largest Number must return "0".

29.5 Same Distance

K Closest Points usually accepts any order among equal distances.

29.6 Stable Order Needed

Use `stable_sort` when equivalent elements should preserve original order.

Example:

Digit logs in Reorder Log Files.

29.7 Large Values

Avoid subtracting in comparator.

Use direct comparisons.

30. Common Mistakes and Wrong Code

Mistake 1 — Using `<=` in Comparator

Wrong:

```
return a[0] <= b[0];
```

Correct:

```
return a[0] < b[0];
```

Mistake 2 — Forgetting Tie-Breaker

Queue Reconstruction needs:

```
height descending, k ascending
```

Without tie-breaker, insertion logic may fail.

Mistake 3 — Wrong Tie Direction

Russian Doll Envelopes needs:

```
width ascending, height descending
```

Not height ascending.

Mistake 4 — Overflow by Subtraction

Wrong:

```
return a[1] - b[1] < 0;
```

Correct:

```
return a[1] < b[1];
```

Mistake 5 — Using Sort Instead of Stable Sort

For Reorder Logs, digit logs must preserve order.

Use:

```
stable_sort
```

Mistake 6 — Comparator Does Not Match Problem Goal

Example:

Largest Number cannot be solved by numeric descending order.

Need:

$a + b > b + a$

Mistake 7 — Confusing Sort Comparator With Priority Queue Comparator

priority_queue comparator has different intuition.

For heap, comparator returns whether one element has lower priority.

31. Interview Explanation Templates

31.1 Sort by One Key

I use a custom comparator to sort by the field that matters for the decision. For example, sorting intervals by end time lets me greedily choose the interval that finishes earliest.

31.2 Sort by Multiple Keys

I first sort by the main key. If there is a tie, I use a secondary key to ensure the order supports the algorithm. This avoids ambiguous ordering and handles equal cases correctly.

31.3 Largest Number

Normal numeric order does not work because the final number depends on concatenation. So for two strings a and b , I compare $a+b$ and $b+a$. If $a+b$ is larger, a should come first.

31.4 Queue Reconstruction

I sort taller people first because they determine the k -count for shorter people. For equal height, I sort by smaller k first. Then inserting each person at index k gives a valid queue.

31.5 Russian Doll Envelopes

I sort width ascending and height descending for equal widths. Then I run LIS on heights. Descending height for equal widths prevents choosing two envelopes with the same width.

32. Complexity Patterns

Sort by one key	$O(n \log n)$	$O(\log n)$ typical
Sort by multiple keys	$O(n \log n)$	$O(\log n)$ typical
Sort by string concatenation	$O(n \log n * m)$	$O(n*m)$
Sort logs	$O(n \log n * m)$	depends on substrings
Sort + insert vector	$O(n^2)$	$O(n)$
Sort + LIS	$O(n \log n)$	$O(n)$
Sort by frequency	$O(n + u \log u)$	$O(u)$
Sort by distance	$O(n \log n)$	$O(1)$ extra

33. Time-Boxed Practice Guidance

Basic Comparator Problems

Target:

10–20 minutes

Examples:

- Sort by distance
- Sort by frequency
- Sort intervals by end

Medium Comparator + Greedy

Target:

20–35 minutes

Examples:

- Non-overlapping Intervals
- Minimum Arrows
- Maximum Units on a Truck
- Queue Reconstruction

Harder Comparator Problems

Target:

35–60 minutes

Examples:

- Largest Number
- Reorder Data in Log Files
- Russian Doll Envelopes
- Top K Frequent Words heap version

34. Expanded Practice Set

34.1 Must-Do Easy / Intro

- 1.1. K Closest Points to Origin, sorting version
 - 2.2. Sort Characters By Frequency
 - 3.3. Maximum Units on a Truck
 - 4.4. Reorder Data in Log Files
 - 5.5. Minimum Absolute Difference
-

34.2 Must-Do Medium

- 6.6. Merge Intervals
 - 7.7. Non-overlapping Intervals
 - 8.8. Minimum Number of Arrows to Burst Balloons
 - 9.9. Largest Number
 - 10.10. Queue Reconstruction by Height
 - 11.11. Top K Frequent Elements
 - 12.12. Top K Frequent Words
 - 13.13. Remove Covered Intervals
 - 14.14. Car Fleet
 - 15.15. Boats to Save People
-

34.3 Strong Interview Practice

- 16.16. Russian Doll Envelopes
 - 17.17. Reorganize String, heap comparator
 - 18.18. Task Scheduler
 - 19.19. Find K Closest Elements
 - 20.20. Hand of Straights
 - 21.21. Array of Doubled Pairs
 - 22.22. Frequency of the Most Frequent Element
 - 23.23. Sort Integers by The Power Value
-

34.4 Hard / Stretch

- 24.24. Russian Doll Envelopes
 - 25.25. The Skyline Problem
 - 26.26. LFU Cache, custom ordering idea in design
 - 27.27. Smallest Range Covering Elements from K Lists
 - 28.28. IPO, heap + custom ordering
 - 29.29. Sliding Window Median, ordered structure comparator logic
-

35. Suggested Study Order

Step 1 — Simple Field Sorting

- 17. K Closest Points
- 18. Maximum Units on a Truck
- 19. Sort Characters By Frequency

Goal:

Learn sorting by computed key or frequency.

Step 2 — Interval Comparator

- 20. Merge Intervals
- 21. Non-overlapping Intervals
- 22. Minimum Arrows
- 23. Remove Covered Intervals

Goal:

Learn start/end/tie-break sorting.

Step 3 — String Comparator

- 24. Largest Number
- 25. Reorder Data in Log Files

Goal:

Learn special string comparison.

Step 4 — Multi-Key Greedy

- 26. Queue Reconstruction by Height
- 27. Russian Doll Envelopes

Goal:

Learn why tie-break direction matters.

Step 5 — Frequency + Ranking

- 28. Top K Frequent Elements
- 29. Top K Frequent Words
- 30. Sort Characters By Frequency

Goal:

Learn frequency maps plus sorting.

36. Pattern Decision Table

sort by interval start	$a[0] < b[0]$
sort by interval end	$a[1] < b[1]$
sort by distance	$\text{distA} < \text{distB}$
frequency descending	$\text{freqA} > \text{freqB}$
frequency desc, lex asc	$\text{freqA} > \text{freqB}$, tie $\text{wordA} < \text{wordB}$
largest concatenation	$a+b > b+a$
queue reconstruction	height desc, k asc
covered intervals	start asc, end desc

Problem Clue	Comparator
envelopes	width asc, height desc
max units	units per box desc
logs	letter before digit, content, id
stable equal elements	use <code>stable_sort</code>

37. Final Cheat Sheet

Sort by Start

```
sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>& b) {
    return a[0] < b[0];
});
```

Sort by End

```
sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>& b) {
    return a[1] < b[1];
});
```

Sort by Two Keys

```
sort(arr.begin(), arr.end(), [](const vector<int>& a, const vector<int>& b) {
    if (a[0] != b[0]) {
        return a[0] < b[0];
    }

    return a[1] > b[1];
});
```

Sort by Distance

```
sort(points.begin(), points.end(), [](const vector<int>& a, const vector<int>& b) {
    long long da = 1LL * a[0] * a[0] + 1LL * a[1] * a[1];
    long long db = 1LL * b[0] * b[0] + 1LL * b[1] * b[1];

    return da < db;
});
```

Largest Number

```
sort(words.begin(), words.end(), [](const string& a, const string& b) {  
    return a + b > b + a;  
});
```

Queue Reconstruction

```
sort(people.begin(), people.end(), [](const vector<int>& a, const vector<int>& b) {  
    if (a[0] != b[0]) {  
        return a[0] > b[0];  
    }  
  
    return a[1] < b[1];  
});
```

Russian Doll Envelopes

```
sort(envelopes.begin(), envelopes.end(), [](const vector<int>& a, const  
vector<int>& b) {  
    if (a[0] != b[0]) {  
        return a[0] < b[0];  
    }  
  
    return a[1] > b[1];  
});
```

Top K Frequent Words Sorting

```
sort(arr.begin(), arr.end(), [](const pair<string, int>& a, const pair<string, int>& b)  
{  
    if (a.second != b.second) {  
        return a.second > b.second;  
    }  
  
    return a.first < b.first;  
});
```

38. What to Master Before Chapter 9

Before moving to Binary Search, make sure you can:

- Write lambda comparators

- Sort by one field
 - Sort by two fields
 - Sort ascending and descending combinations
 - Avoid `<=` in comparator
 - Avoid subtraction overflow in comparator
 - Explain strict weak ordering
 - Sort intervals by start/end
 - Know why tie-breakers matter
 - Solve Largest Number comparator
 - Solve Queue Reconstruction comparator
 - Solve Russian Doll Envelopes sorting
 - Know when to use `stable_sort`
 - Understand how custom comparator supports greedy logic
-

39. Core Takeaway

A custom comparator is how you tell C++:

This is the order that matters for my problem.

The most important question is:

What should come first, and why?

If you can answer that, you can write the comparator.

Common comparator patterns:

```
a[0] < b[0]           // start ascending
a[1] < b[1]           // end ascending
a[1] > b[1]           // value descending
a + b > b + a         // largest concatenation
height desc, k asc    // queue reconstruction
width asc, height desc // envelope/LIS trick
```

Comparator problems often test not just coding, but reasoning.

The sort order must support the later algorithm.

A wrong comparator usually means the whole solution fails, even if the rest of the code looks correct.
